Diseño y Análisis de Algoritmos

Ayudantía 6: Algoritmos Voraces

Ayud. Ferran Rojas Andreu

1. (a) Criterio voraz y pseudocódigo

Idea. Ordenar las solicitudes por *tiempo de término* creciente y aceptar cada solicitud si es compatible con la última aceptada (es decir, si su inicio es mayor o igual que el término de la última elegida).

IntervalScheduling(R)

```
1  Ordenar R por f(i) no decreciente
2  A ← ∅;      last ← −∞
3  for cada solicitud i en ese orden
4      if s(i) ≥ last
5          A ← A ∪ {i}
6          last ← f(i)
7  return A
```

2. (b) Ejecución sobre la instancia

Las solicitudes ya están ordenadas por término f:

$$(1,3),\ (2,5),\ (4,6),\ (5,7),\ (3,8),\ (7,9).$$

Solicitud	$s \geq last$?	Decisión	Conjunto A
$[1,3)$	$1 \geq -\infty$	**aceptar**	$\{[1,3)\}$, $last = 3$
$[2,5)$	$2 \geq 3$? No	descartar	$\{[1,3)\}$
$[4,6)$	$4 \geq 3$? Sí	**aceptar**	$\{[1,3),[4,6)\}$, $last = 6$
$[5,7)$	$5 \geq 6$? No	descartar	$\{[1,3),[4,6)\}$
$[3,8)$	$3 \geq 6$? No	descartar	$\{[1,3),[4,6)\}$
$[7,9)$	$7 \geq 6$? Sí	**aceptar**	$\{[1,3),[4,6),[7,9)\}$, $last = 9$

Solución del algoritmo:

$$\boxed{A = \{[1,3),\ [4,6),\ [7,9)\}, \qquad |A| = 3}.$$

¿Es óptimo? Con 6 intervalos no es posible seleccionar 4 mutuamente compatibles: el intervalo $[1,3)$ o $[2,5)$ bloquea el extremo izquierdo, y los intervalos centrales se solapan entre sí. De hecho, una solución óptima también tiene tamaño 3 (por ejemplo $\{[2,5),[5,7),[7,9)\}$). Luego A es óptimo.

3. (c) Prueba de optimalidad por argumento de intercambio

Sea $G = \{g_1, \ldots, g_k\}$ la solución voraz (con $g_1, \ldots, g_k$ en el orden en que fueron aceptados) y sea $O = \{o_1, \ldots, o_m\}$ una solución óptima cualquiera, también ordenada por tiempo de término.

Afirmación 1. Para todo j, se cumple $f(g_j) \leq f(o_j)$.

Demostración por inducción. Para $j = 1$, g_1 es la solicitud de menor término entre todas, así que $f(g_1) \leq f(o_1)$. Supongamos $f(g_j) \leq f(o_j)$. Como g_{j+1} es la primera solicitud compatible con g_j (i.e., con $s \geq f(g_j)$), se tiene $s(g_{j+1}) \leq s(o_{j+1})$ o, más precisamente, g_{j+1} termina antes que cualquier solicitud compatible con g_j; en particular, como $f(o_j) \geq f(g_j)$, la solicitud o_{j+1} (que cumple $s(o_{j+1}) \geq f(o_j) \geq f(g_j)$) es compatible con g_j, por lo que el voraz la habría considerado antes: $f(g_{j+1}) \leq f(o_{j+1})$. □

Afirmación 2. $|G| \geq |O|$.

Demostración. Supongamos que $|O| > |G|$, es decir $m \geq k + 1$. Por la Afirmación 1, $f(g_k) \leq f(o_k)$. Como o_{k+1} existe y $s(o_{k+1}) \geq f(o_k) \geq f(g_k)$, la solicitud o_{k+1} es compatible con g_k. Pero entonces el algoritmo voraz, tras aceptar g_k, habría aceptado una solicitud compatible más (al menos o_{k+1}), contradicción con que g_k fue la última aceptada. □

Como $|G| \geq |O|$ y O es óptima, G también es óptima. ■

4. (d) Complejidad

Ordenar las n solicitudes por término cuesta $O(n \log n)$. El barrido posterior recorre las n solicitudes realizando trabajo constante por cada una, $O(n)$. Por lo tanto, el tiempo total es

$$\boxed{\mathcal{O}(n \log n)}.$$

5. (a) Criterio voraz y pseudocódigo

Idea. Ordenar los niños por glotonería creciente y las galletas por tamaño creciente. Luego, en orden, al niño actual se le asigna la galleta *más pequeña que lo conforme*: si la galleta apuntada alcanza ($s[j] \geq g[i]$), el niño queda conforme y avanzan ambos punteros; si no alcanza, esa galleta no le servirá a ningún niño posterior (todos son al menos tan glotones), así que se descarta.

AssignCookies($g[1..n]$, $s[1..k]$)

```
1  Ordenar g y s en orden no decreciente
2  i ← 1; j ← 1; conten ← 0
3  while j ≤ k and i ≤ n
4      if s[j] ≥ g[i]
5          conten ← conten + 1; i ← i + 1; j ← j + 1
6      else
7          j ← j + 1                ▷ la galleta j no sirve para el niño menos glotón restante
8  return conten
```

Equivalencia con el código de LeetCode. La solución C++ del enunciado (problema 455) es la misma estrategia con dos punteros p (galleta) y p_2 (niño); el `while` interno del `else` no es más que el paso $j \leftarrow j + 1$ de arriba repetido para descartar todas las galletas demasiado pequeñas.

6. (b) Ejecución sobre $g = \langle 3, 1, 2 \rangle$, $s = \langle 2, 1, 1, 3 \rangle$

Ordenados: $g = \langle 1, 2, 3 \rangle$, $s = \langle 1, 1, 2, 3 \rangle$. Estado de los punteros i (niño), j (galleta) y del contador:

i (niño $g[i]$)	j (galleta $s[j]$)	Acción
1 (g=1)	1 (s=1)	$1 \geq 1$ **conforme**: i=2, j=2, *conten*=1
2 (g=2)	2 (s=1)	$1 < 2$: descartar galleta, j=3
2 (g=2)	3 (s=2)	$2 \geq 2$ **conforme**: i=3, j=4, *conten*=2
3 (g=3)	4 (s=3)	$3 \geq 3$ **conforme**: i=4, j=5, *conten*=3

Se agotaron las galletas y terminó el bucle:

$$\boxed{\text{niños conformes } = 3}.$$

Efectivamente los tres niños pueden quedar conformes (p. ej. niño glotón 1 con galleta 1, niño glotón 2 con galleta 2 y niño glotón 3 con galleta 3). ✓

7. (c) Prueba de optimalidad por argumento de intercambio

Sean los niños ordenados por glotonería $g[1] \leq g[2] \leq \cdots \leq g[n]$ y las galletas por tamaño $s[1] \leq \cdots \leq s[k]$. Sea s^* la galleta más pequeña con $s^* \geq g[1]$.

Afirmación (propiedad de la elección voraz). Existe una solución óptima que asigna s^* al niño de menor glotonería $g[1]$.

Demostración. Sea O una solución óptima que conforme al niño $g[1]$ (si no lo conforme, se le puede dar la galleta de cualquier niño conforme, que también lo satisface por ser el menos

glotón, sin perder conformes). En O, el niño $g[1]$ recibe una galleta $c \geq g[1]$; como s^* es la menor galleta con $s^* \geq g[1]$, se tiene $s^* \leq c$.

- Si s^* no se usa en O, reemplazar c por s^* en O conserva el número de conformes.
- Si s^* se usa para otro niño j en O, entonces $g[j] \leq s^*$ (ese niño está conforme). Intercambiando las galletas de ambos niños: $g[1]$ recibe s^* (vale, pues $s^* \geq g[1]$) y el niño j recibe c. Como $g[j] \leq s^* \leq c$, el niño j sigue conforme.

En ambos casos se obtiene una solución óptima O' que asigna s^* al niño $g[1]$. Además, todas las galletas menores que s^* no satisfacen a nadie (por definición de s^*) y pueden descartarse, igual que hace el voraz. □

Optimalidad por inducción. Por la Afirmación, la decisión voraz es *segura*: hay una óptima que la incluye. Eliminados el niño $g[1]$ y la galleta s^*, el resto es un subproblema del mismo tipo (misma estructura), y por inducción el voraz es óptimo en él. Por lo tanto, el algoritmo devuelve el máximo número de niños conformes. ■

8. (d) Complejidad

Ordenar ambos arreglos cuesta $O(n \log n + k \log k)$. El barrido con dos punteros recorre a lo más $n + k$ posiciones realizando trabajo constante por paso:

$$\boxed{\mathcal{O}(n \log n + k \log k)}$$

tiempo y $\mathcal{O}(1)$ espacio adicional (sin contar el ordenamiento).